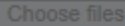


[1]  !pip install -q transformers torch accelerate pandas pyarrow

[7] 
import torch
import pandas as pd
import re
from transformers import AutoTokenizer, AutoModelForCausalLM
from google.colab import files

[9]  9s
uploaded = files.upload()

▼  train-00000-...001.parquet
train-00000-of-00001.parquet(n/a) - 2306545 bytes, last modified: 25/02/2026 - 100% done
Saving train-00000-of-00001.parquet to train-00000-of-00001.parquet

[10]  0s
file_name = list(uploaded.keys())[0]
data = pd.read_parquet(file_name)

print("Total samples:", len(data))
data.head()

▼ Total samples: 7473

	question	answer
0	Natalia sold clips to 48 of her friends in Apr...	Natalia sold $48/2 = \ll 48/2=24 \gg$ 24 clips in May...
1	Weng earns \$12 an hour for babysitting. Yester...	Weng earns $12/60 = \$\ll 12/60=0.2 \gg$ 0.2 per minut...
2	Betty is saving money for a new wallet which c...	In the beginning, Betty has only $100 / 2 = \$\ll \dots$
3	Julie is reading a 120-page book. Yesterday, s...	Maila read $12 \times 2 = \ll 12*2=24 \gg$ 24 pages today....
4	James writes a 3-page letter to 2 different fr...	He writes each friend $3*2=\ll 3*2=6 \gg$ 6 pages a w...

Next steps: [Generate code with data](#) [New interactive sheet](#)

[11]  0s
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)

▼ Using device: cuda

[12]  37s
model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForCausalLM.from_pretrained(
 model_name,
 torch_dtype=torch.float16 if device == "cuda" else torch.float32,
 low_cpu_mem_usage=True
) .to(device)

config.json: 100%  608/608 [00:00<00:00, 64.7kB/s]

tokenizer_config.json:  1.29k/? [00:00<00:00, 109kB/s]

tokenizer.json:  1.84M/? [00:00<00:00, 65.9MB/s]

tokenizer.model: 100%  500k/500k [00:01<00:00, 397kB/s]

special_tokens_map.json: 100%  551/551 [00:00<00:00, 48.5kB/s]

model.safetensors: 100%  2.20G/2.20G [00:24<00:00, 193MB/s]

Loading weights: 100%  201/201 [00:08<00:00, 23.05it/s, Materializing param=model.norm.weight]

generation_config.json: 100%  124/124 [00:00<00:00, 16.0kB/s]

```
LlamaForCausalLM(  
  (model): LlamaModel(  
    (embed_tokens): Embedding(32000, 2048)  
    (layers): ModuleList(  
      (0-21): 22 x LlamaDecoderLayer(  
        (self_attn): LlamaAttention(  
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)  
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)  
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)  
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)  
        )  
        (mlp): LlamaMLP(  
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)  
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)  
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)  
          (act_fn): SiLUActivation()  
        )  
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)  
      )  
    )  
    (norm): LlamaRMSNorm((2048,), eps=1e-05)  
    (rotary_emb): LlamaRotaryEmbedding()  
  )  
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)  
)
```

```
def build_prompt(question, mode="zero_shot", exemplars=None):
```

```
    if mode == "zero_shot":
```

```
        return (
```

```
            "Solve the following math problem.\n"
```

```
            "Give only the final numeric answer.\n\n"
```

```
            f"Question: {question}\n"
```

```
            "Answer:"
```

```
        )
```

```
    elif mode == "few_shot":
```

```
        prompt = "Here are some solved examples:\n\n"
```

```
        for q, a in exemplars:
```

```
            prompt += f"Q: {q}\nA: {a}\n\n"
```

```
        prompt += (
```

```
            "Now solve the following problem.\n"
```

```
            f"Q: {question}\n"
```

```
            "A:"
```

```
        )
```

```
        return prompt
```

```
    elif mode == "cot":
```

```
        return (
```

```
            "Solve the following math problem step by step.\n"
```

```
            "After reasoning, give the final answer in the format:\n"
```

```
            "Answer: <number>\n\n"
```

```
            f"Question: {question}\n"
```

```
            "Let's think step by step."
```

```
        )
```

```
def generate_answer(prompt, max_new_tokens=128):

    inputs = tokenizer(prompt, return_tensors="pt").to(device)

    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=max_new_tokens,
            do_sample=False
        )

    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

```
def extract_last_number(text):
    numbers = re.findall(r"-?\d+\.?\d*", text)
    return numbers[-1] if numbers else None
```

```
def get_few_shot_examples(k=3):

    examples = []

    for i in range(k):
        q = data.iloc[i]["question"]
        a = data.iloc[i]["answer"].split("####")[-1].strip()
        examples.append((q, a))

    return examples
```

```
few_shot_examples = get_few_shot_examples(k=3)
```

```
def evaluate_mode(mode, n_samples=50):
```

```
    correct = 0
```

```
    hallucinations = 0
```

```
    for i in range(n_samples):
```

```
        question = data.iloc[i]["question"]
```

```
        gt_answer = data.iloc[i]["answer"].split("####")[-1].strip()
```

```
        if mode == "few_shot":
```

```
            prompt = build_prompt(question, mode, few_shot_examples)
```

```
        else:
```

```
            prompt = build_prompt(question, mode)
```

```
        output = generate_answer(prompt)
```

```
        pred = extract_last_number(output)
```

```
        gt = extract_last_number(gt_answer)
```

```
        if pred is None:
```

```
            hallucinations += 1
```

```
        elif pred == gt:
```

```
            correct += 1
```

```
        if i % 10 == 0:
```

```
            print(f"[{mode}] Processed {i+1}/{n_samples}")
```

```
    accuracy = correct / n_samples
```

```
    hallucination_rate = hallucinations / n_samples
```

```
    print("\n-----")
```

```
    print(f"Mode: {mode}")
```

```
    print(f"Accuracy: {accuracy:.3f}")
```

```
    print(f"Hallucination Rate: {hallucination_rate:.3f}")
```

```
    print("-----\n")
```

```
    return accuracy, hallucination_rate
```

```
acc_zero, hall_zero = evaluate_mode("zero_shot")
acc_few, hall_few = evaluate_mode("few_shot")
acc_cot, hall_cot = evaluate_mode("cot")
```

```
[zero_shot] Processed 1/50
[zero_shot] Processed 11/50
[zero_shot] Processed 21/50
[zero_shot] Processed 31/50
[zero_shot] Processed 41/50
```

```
-----
Mode: zero_shot
Accuracy: 0.020
Hallucination Rate: 0.040
-----
```

```
[few_shot] Processed 1/50
[few_shot] Processed 11/50
[few_shot] Processed 21/50
[few_shot] Processed 31/50
[few_shot] Processed 41/50
```

```
-----
Mode: few_shot
Accuracy: 0.000
Hallucination Rate: 0.000
-----
```

```
[cot] Processed 1/50
[cot] Processed 11/50
[cot] Processed 21/50
[cot] Processed 31/50
[cot] Processed 41/50
```

```
-----
Mode: cot
Accuracy: 0.020
Hallucination Rate: 0.000
-----
```